



# Erros Defeitos e Falhas

**U**m dos objetivos de melhorar o processo de desenvolvimento de software é minimizar o efeito de defeitos. Desta forma, convém entendermos melhor o que causam defeitos em produtos de software e o que podemos fazer para melhorá-los.

Muitas vezes ouvimos falar de erros, defeitos e falhas, talvez utilizando-os como sinônimos. Contudo, segundo a norma IEEE 610.12/1990, temos definições diferentes para erros, defeitos e falhas. Os problemas introduzidos no software durante seu processo de desenvolvimento são chamados de falhas, sejam eles próprios da especificação ou da implementação do sistema. A falha pode ser ativada durante a execução do software e gerar um erro. Quando o erro é percebido pelo usuário, então temos um defeito (PEREZ et al., 2006).

Desta forma, podemos compreender que a causa dos defeitos é, de forma resumida, as falhas no processo de desenvolvimento. Afinal de contas, falhas geram erros, que se percebidos pelo usuário final, resultam em defeitos. O que causa estas falhas em softwares modernos, e por que os sistemas aparentam ter cada vez mais erros?

Primeiro, podemos pensar que há uma complexidade crescente dos sistemas. Com as novas tecnologias e sistemas cada vez maiores e mais integrados, além da pressão de desenvolver e lançar software de forma cada vez mais rápida, é possível que o processo de desenvolvimento não consiga prever todos os casos possíveis de erros durante a integração com outros sistemas (GALLOTTI, 2015).

Outra possível causa de falhas é o não uso de métodos de Engenharia de Software, boas práticas e outros conhecimentos existentes no desenvolvimento profissional de software, o que pode causar instabilidades nos softwares resultantes (GALLOTTI, 2015).

Para encontrar e reportar um defeito, podemos utilizar o recurso de inspeção, que são revisões em pares que possuem como objetivo identificar e remover defeitos. É comum que membros da equipe de desenvolvimento se organizem para inspecionar o software em desenvolvimento para encontrar erros e falhas (GALLOTTI, 2015).

Esta inspeção pode ser executada no próprio código fonte, de forma que cada inspetor observa o código linha por linha com o objetivo de identificar erros lógicos, anomalias, condições erradas e recursos omitidos (GALLOTTI, 2015).

Tão importante quanto identificar algum defeito é descrevê-lo de forma efetiva. Para isso, os inspetores usam um checklist que organiza e padroniza esta descrição. Particularidades desta lista podem estar relacionados ao projeto e linguagem utilizada. Os resultados da inspeção são apresentados em uma reunião, quando é decidido se o material do projeto é (GALLOTTI, 2015):

- Material aceito sem alterações, de forma integral;
- Material aceito com poucas alterações, com revisões técnicas dispensáveis, sem necessidade de nova inspeção;
- Material rejeitado e com muitas alterações necessárias, com necessidade de nova inspeção;

- Material rejeitado e enviado para reconstrução;
- Revisão incompleta, com necessidade de finalizar a inspeção em outro momento.

Uma questão importante no processo de inspeção é nunca julgar ao reportar defeitos. Principalmente, é importante não julgar pessoas ao reportar defeitos. De referência, a inspeção deve ser um processo técnico, cujas decisões sejam pautadas apenas em fatos sobre o artefato em si, e não no autor do artefato. Inibir processos de inspeção, identificação e resolução de problemas podem ser resultados negativos de dinâmicas organizacionais nocivas, que podem ser como consequência uma qualidade pior do produto de software. Uma sugestão é lembrar que a inspeção busca identificar problemas no software, e não do seu autor.

Outra questão é a possibilidade de isolar e reproduzir defeitos. Uma descrição efetiva de defeitos permite que a equipe de desenvolvimento consiga reproduzir o defeito, se este for encontrado durante a execução do código. Sem conseguir reproduzir o defeito ou identificar em que linha do código ele está, a equipe pode ter muita dificuldade em corrigir algum defeito identificado.

Por fim, o software pode se prevenir contra falhas de outros sistemas da informação ao implementar mecanismos de tolerância a falhas, de forma a evitar que falhas se transformem em defeitos. Por exemplo, mesmo que um sistema externo esteja indisponível em determinado momento, um sistema tolerante a falhas pode utilizar um sistema redundante no lugar do sistema externo indisponível. Desta forma, mesmo que tenha ocorrido uma falha, ela não causa um defeito. Neste caso, o usuário nem percebe que a falha de um sistema externo ocorreu.

## Atividade Extra

Pesquise defeitos de algum sistema da informação utilizado em larga escala, como alguma rede social, e reflita como estes defeitos poderiam ser descritos de forma efetiva.

## Referências Bibliográficas

PEREZ, Ivan Rodolfo Duran Cruz et al. **Um processo para testes de sistemas com reuso de componentes**. Technical Report IC-06-21, 2006.

GALLOTTI, G. M. A. (Org.). **Qualidade de software**. Pearson: 2015.

**Ir para exercício**